Компилятор Go под капотом

От одной строки Go-кода до IR, SSA, вызовов рантайма и машинных инструкций.

```
синтаксис → доказательства → SSA
путь через рантайм или прямо в код
amd64 / arm64 → машинные инструкции
```

Чего не хватало после 4.1 и 4.2

ЧТО УЖЕ ЗНАЕМ

Мы знаем, как работают `growslice`, Swiss Table, `ITab`, `chansend` и `selectgo`.

НОВЫЙ ВОПРОС

Но не выяснили, кто решил, что `append`, `m[key]` и `ch <- v` превратятся именно в эти действия.

ИНЖЕНЕРНЫЙ ВЫВОД

Этот недостающий слой — компилятор, а не еще один алгоритм рантайма.

Строка исходника не равна инструкции CPU

УПРОЩЕНИЕ

`x := a + b` иногда действительно близка к одной операции сложения.

ТОЧНАЯ МОДЕЛЬ

`append(s, x)` включает проверку емкости, ветку роста, адресную арифметику и запись элемента.

ИНЖЕНЕРНЫЙ ВЫВОД

Физическую цену определяет код после оптимизаций, а не длина исходной строки.

Пять знакомых конструкций

GO / РАНТАЙМ

```
s = append(s, x)
v := m[key]
w.Write(buf)
ch <- v
select { case <-a: case <-b: }
```

ЧТО ПРОИСХОДИТ

Компилятор может оставить прямые операции, выбрать вспомогательную функцию рантайма, убрать косвенный вызов или упростить весь `select`.

ИНЖЕНЕРНЫЙ ВЫВОД

Синтаксис языка не определяет единственный путь исполнения.

Маршрут этой лекции

ПОЛНАЯ ЦЕПОЧКА

```
исходник Go → синтаксис / типы
IR → инлайнинг / девиртуализация / escape
walk → SSA → доказательства
lower → регистры / стек → машинный код
```

ИНЖЕНЕРНЫЙ ВЫВОД

В конце каждый шаг проверим на знакомых конструкциях.

Что намеренно не изучаем

ГРАНИЦА

Не строим собственный парсер, не доказываем теорию доминаторов и не изучаем все правила архитектурного ABI.

ЦЕЛЬ

Концентрируемся на механизмах, которые меняют стоимость горячего backend-кода.

ИНЖЕНЕРНЫЙ ВЫВОД

После каждого внутреннего механизма возвращаемся к наблюдаемому коду и профилю.

От текста к внутреннему представлению

Сначала понять программу, затем доказать свойства, потом выбрать операции CPU.

```
синтаксис / проверка типов
Unified IR / middle end / walk
SSA / lower / машинный код
```

Этапы компиляции: верхняя половина

ДО SSA

```
.go → разбор текста и синтаксиса
→ проверка типов
→ внутреннее IR / Unified IR
→ middle end: инлайнинг, девиртуализация, escape
→ walk: упрощение конструкций
```

ИНЖЕНЕРНЫЙ ВЫВОД

На верхних этапах компилятор еще работает со смыслом Go-программы.

Этапы компиляции: нижняя половина

ПОСЛЕ SSA

```
SSA: значения + блоки + память
→ анализ и переписывания
→ lower: операции amd64 / arm64
→ распределение регистров и стековый кадр
→ obj.Prog → ассемблер → машинный код
```

ИНЖЕНЕРНЫЙ ВЫВОД

Ближе к CPU уменьшается доля синтаксиса и растет доля ограничений архитектуры.

Парсер сначала восстанавливает структуру

GO / РАНТАЙМ

```
result := a + b*c
```

ЧТО ПРОИСХОДИТ

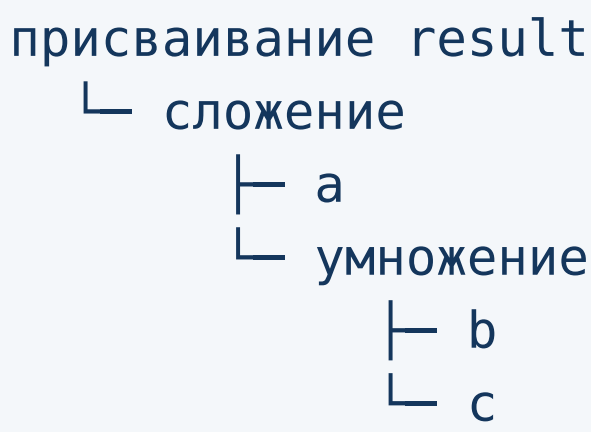
Выражение — это сложение `a` с произведением `b*c`. Порядок задается деревом синтаксиса, а не текстовым порядком символов.

ИНЖЕНЕРНЫЙ ВЫВОД

Разбор синтаксиса отвечает «что написано», а не «как быстро выполнить».

Синтаксическое дерево

СТРУКТУРА



ИНЖЕНЕРНЫЙ ВЫВОД

Структура нужна еще до выбора конкретных низкоуровневых операций.

Почему одного AST недостаточно

СИНТАКСИС

AST: вызов, оператор, ветка, цикл; хорошо описывает форму исходника.

ОПТИМИЗАЦИИ

Внутреннее IR должно хранить типы, разрешенные вызовы и факты для оптимизаций.

ИНЖЕНЕРНЫЙ ВЫВОД

Синтаксическое дерево и IR решают разные задачи.

Проверка типов меняет смысл операции

GO / РАНТАЙМ

```
var a int32
var b int64
// a + b  // требуется
согласовать типы
```

ЧТО ПРОИСХОДИТ

\+ для чисел и строк — разные механизмы. Компилятор должен знать тип и размер операндов до генерации кода.

ИНЖЕНЕРНЫЙ ВЫВОД

Оператор в тексте один, набор допустимых машинных действий зависит от типов.

Тип уже влияет на стоимость

ФАКТ

Компилятор знает размер, выравнивание, знаковость и представление типа.

СЛЕДСТВИЕ

Поэтому заранее выбирает допустимые низкоуровневые операции и правила работы с памятью.

ИНЖЕНЕРНЫЙ ВЫВОД

Система типов — не только проверка корректности API.

От проверенных типов к Unified IR

ЗАДАЧА

После проверки типов текущий `cmd/compile` переводит программу к собственным типам и IR. В описании компилятора фигурирует этап `noding`.

МЕХАНИЗМ

Теперь оптимизации получают пригодную для преобразований модель программы.

ИНЖЕНЕРНЫЙ ВЫВОД

Имена промежуточных этапов относятся к реализации Go 1.27.x.

Почему имена локальных переменных исчезают

УПРОЩЕНИЕ

В исходнике: `x := f(); y := x + 1; return y`.

ТОЧНАЯ МОДЕЛЬ

После оптимизаций `x` и `y` могут быть заменены значениями и вовсе не получить физического места.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор сохраняет семантику, а не структуру переменных из текста.

Middle end — слой доказательств

ЗАДАЧА

Компилятор уже знает типы и структуру. Теперь ищет недостижимые ветки, известные типы receiver и места, где функция может быть встроена.

МЕХАНИЗМ

Эти факты будут менять следующие этапы: вызовы, аллокации, проверки.

ИНЖЕНЕРНЫЙ ВЫВОД

Оптимизации не просто ускоряют операции — они меняют вид программы.

Мертвая ветка может исчезнуть

GO / РАНТАЙМ

```
if false {
    expensive()
}
```

ЧТО ПРОИСХОДИТ

При известном условии компилятор удаляет недостижимый путь. Его нет в рабочем машинном коде.

ИНЖЕНЕРНЫЙ ВЫВОД

Наличие строки в исходнике не гарантирует наличия соответствующих инструкций.

Компилятор не обязан сохранить вызов функции

ЗАДАЧА

Маленькую функцию можно встроить (inlining) в вызывающую.

МЕХАНИЗМ

После этого результаты ее вычислений могут стать константами; иногда исчезает и сама операция.

ИНЖЕНЕРНЫЙ ВЫВОД

Главная ценность инлайнинга часто в новых возможностях доказать свойства кода.

Инлайнинг открывает новые факты

GO / РАНТАЙМ

```
func add(a, b int) int {
    return a + b
}

func f() int { return add(10, 20)
}
```

ЧТО ПРОИСХОДИТ

Вызов может превратиться сначала в `10 + 20`, затем в `20+10 = 30` как константный результат.

ИНЖЕНЕРНЫЙ ВЫВОД

После инлайнинга возможны распространение констант и удаление мертвого кода.

Оптимизации усиливают друг друга

ПРИЧИННАЯ ЦЕПОЧКА

```
известный конкретный тип receiver
↓ девиртуализация
прямой вызов
↓ инлайнинг
тело функции у вызывающей
↓ константы / escape / проверки
```

ИНЖЕНЕРНЫЙ ВЫВОД

Экономия одного косвенного перехода может открыть намного более крупное улучшение.

Результат эскейп-анализа зависит от окружения

ЗАДАЧА

В предыдущей лекции мы обсуждали stack/heap. Но отдельная функция не всегда определяет стоимость аллокации сама по себе.

МЕХАНИЗМ

Инлайнинг раскрывает время жизни и места использования значений; решение может поменяться.

ИНЖЕНЕРНЫЙ ВЫВОД

Проверяем `-m=2` на конкретной сборке, а не делаем вывод по синтаксису.

Как исчезает синтаксический сахар

Код должен сохранить порядок побочных эффектов, но стать проще для SSA.

```
порядок вычислений
операции map / channel / select
встроенный путь + помощь рантайма
```

Зачем нужен `walk`

ЗАДАЧА

После ранних оптимизаций остается множество высокоуровневых конструкций Go.

МЕХАНИЗМ

`walk` фиксирует необходимый порядок вычислений и превращает сложные формы в более примитивные операции.

ИНЖЕНЕРНЫЙ ВЫВОД

Следующие фазы не обязаны понимать каждую языковую конструкцию целиком.

Порядок вычислений — часть семантики

GO / РАНТАЙМ

```
dst[f()] = g()
```

ЧТО ПРОИСХОДИТ

Вызовы `f()` и `g()` могут иметь побочные эффекты. Компилятор не вправе произвольно поменять наблюдаемый порядок.

ИНЖЕНЕРНЫЙ ВЫВОД

Упрощение не должно менять поведение программы.

От операции map к понятным шагам

ПРЕОБРАЗОВАНИЕ

```
v := m[key]
↓ вычислить map и key
↓ выбрать типизированный путь
↓ вызвать вспомогательную функцию рантайма
↓ получить результат
```

ИНЖЕНЕРНЫЙ ВЫВОД

Высокоуровневый индекс `m[key]` не сохраняется до CPU как отдельный opcode.

Встроенный путь и вызов рантайма

УПРОЩЕНИЕ

Неудачная модель: любой `append` полностью передается функции рантайма.

ТОЧНАЯ МОДЕЛЬ

Компилятор может встроить проверку емкости и запись, а `runtime.growslice` вызывать только при росте.

ИНЖЕНЕРНЫЙ ВЫВОД

Код после `walk` часто состоит из частого быстрого и редкого медленного пути.

Переход от Go к операциям рантайма

МОСТ К РАНТАЙМУ

```
append → проверка len/cap → growslice при необходимости
m[key] → mapaccess* / mapassign*
ch <- v → chansend
select → специальная форма или selectgo
```

ИНЖЕНЕРНЫЙ ВЫВОД

Только после выбора пути начинаются механизмы из лекций 4.1 и 4.2.

Одно присваивание, явные зависимости

Как представить вычисления так, чтобы оптимизатор мог надежно доказывать свойства.

```
SSA Value / Block
Phi / управление
Memory / побочные эффекты
```

SSA: одно значение определяется один раз

GO / РАНТАЙМ

```
x := 1
x = x + 2
x = x * 10
```

ЧТО ПРОИСХОДИТ

Учебная запись: `x1 = 1`; `x2 = x1 + 2`; `x3 = x2 * 10`. У каждого значения единственное определение.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятору легче понять зависимости между результатами.

Обычная переменная и SSA-значение

УПРОЩЕНИЕ

`x` в Go может многократно менять значение; нужно отслеживать присваивания.

ТОЧНАЯ МОДЕЛЬ

`x1`, `x2`, `x3` — разные вычисления; ссылки явно указывают, какое значение используется.

ИНЖЕНЕРНЫЙ ВЫВОД

SSA — граф вычислений, а не Go с переименованными локальными переменными.

SSA `Value`

ПРЕДСТАВЛЕНИЕ

Например: `v4 = Add64 <int> v2 v3`.

СМЫСЛ

Значение знает операцию, тип и аргументы. Дальнейшие проходы могут его заменить или удалить.

ИНЖЕНЕРНЫЙ ВЫВОД

`Value` не равно физическому регистру или локальной переменной.

SSA выглядит уже не как Go

GO / РАНТАЙМ

```
v4 = Add64 <int> v2 v3
```

ЧТО ПРОИСХОДИТ

Это описание вычисления, а не готовая инструкция `ADDQ BX, AX`. Конкретная ISA будет выбрана позже.

ИНЖЕНЕРНЫЙ ВЫВОД

Низкоуровневые операции сначала остаются архитектурно-независимыми.

Функция — не просто список вычислений

ЗАДАЧА

В Go есть `if`, `for`, `return` и разные пути исполнения.

МЕХАНИЗМ

SSA организует управление в базовые блоки (`Block`) и связи между ними.

ИНЖЕНЕРНЫЙ ВЫВОД

Оптимизации анализируют не только значения, но и то, по каким веткам можно дойти до них.

Базовый блок: вход и выход

GO / РАНТАЙМ

```
if x > 0 {
    f()
} else {
    g()
}
```

ЧТО ПРОИСХОДИТ

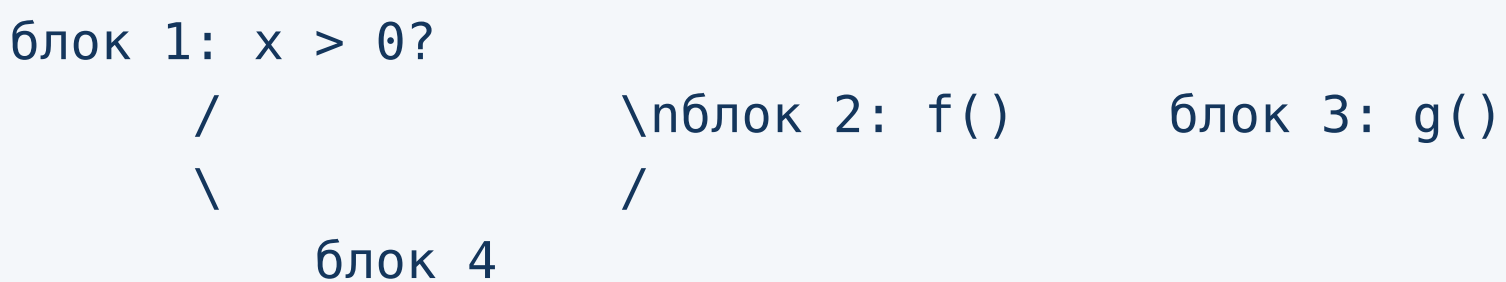
Граф: проверка блок `f()` или `g()` общая точка. Внутри блока нет произвольного прыжка в середину.

ИНЖЕНЕРНЫЙ ВЫВОД

Граф управления показывает, какие операции могут выполняться вместе.

`if` как граф управления

БЛОКИ



ИНЖЕНЕРНЫЙ ВЫВОД

Имена исходных веток уже не нужны: остаются блоки и переходы.

Слияние значений после ветвления

GO / РАНТАЙМ

```
if cond {
    x = 10
} else {
    x = 20
}
return x + 1
```

ЧТО ПРОИСХОДИТ

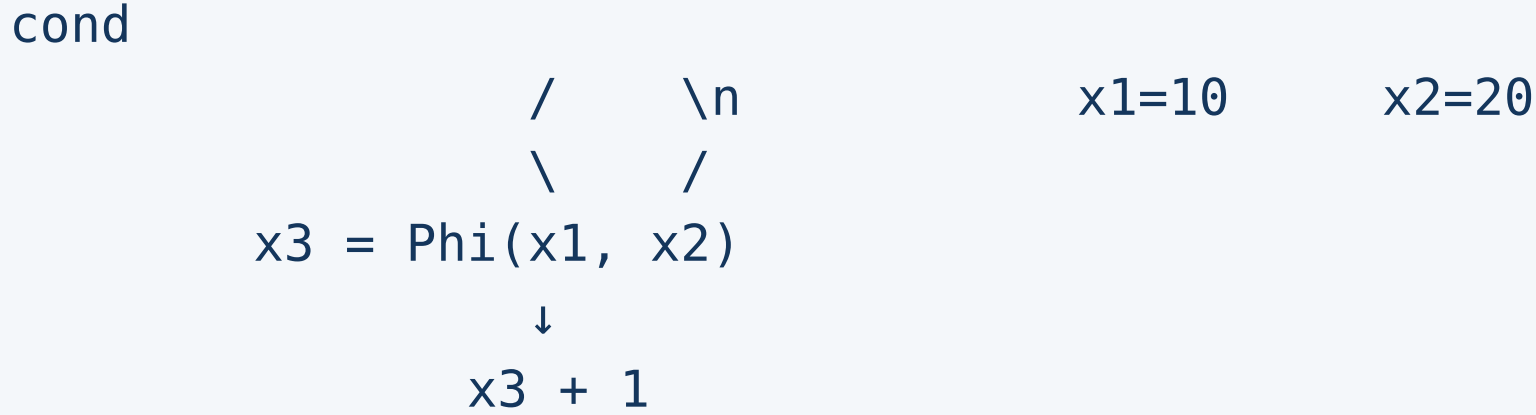
У SSA должны быть два значения веток и одно значение в месте слияния.

ИНЖЕНЕРНЫЙ ВЫВОД

Простого переименования переменных недостаточно.

Phi: откуда пришло управление?

СЛИЯНИЕ



ИНЖЕНЕРНЫЙ ВЫВОД

`Phi` выбирает значение по предшествующему блоку, а не по условию «из воздуха».

`Phi` — не машинная инструкция

SSA

На этапе SSA есть `Phi(x1, x2)`.

МАШИННЫЙ КОД

При генерации кода компилятор реализует выбор через размещение/перемещение значений и пути управления. Инструкции `PHI` у CPU нет.

ИНЖЕНЕРНЫЙ ВЫВОД

Промежуточное представление описывает зависимости, не набор opcode.

SSA позволяет распространять константы

ЗАДАЧА

`v2 = Const64 10`; `v3 = Const64 20`; `v4 = Add64 v2 v3`.

МЕХАНИЗМ

Результат `v4 = Const64 30` можно доказать заранее; операция сложения исчезает.

ИНЖЕНЕРНЫЙ ВЫВОД

Удаление инструкции — следствие доказательства, а не нарушение семантики.

SSA позволяет удалить неиспользуемое значение

ЗАДАЧА

Если v7 больше никем не используется и не имеет побочных эффектов, хранить его в графе незачем.

МЕХАНИЗМ

Проход удаления мертвого кода выбрасывает это вычисление.

ИНЖЕНЕРНЫЙ ВЫВОД

Наличие выражения в исходнике не означает работу CPU.

Как SSA учитывает изменения памяти

Значения в SSA определяются однажды. А записи по указателю меняют одни и те же байты.

```
Memory как SSA-значение
Load / Store / Call
порядок побочных эффектов
```

Проблема: память допускает повторную запись

GO / РАНТАЙМ

```
*p = 10
*p = 20
```

ЧТО ПРОИСХОДИТ

Если две операции считать независимыми, компилятор мог бы неправильно поменять их местами.

ИНЖЕНЕРНЫЙ ВЫВОД

Для памяти в SSA нужна отдельная цепочка зависимостей.

Состояние памяти — тоже значение SSA

MEMORY

```
mem0
  ↓ Store(p, 10, mem0)
mem1
  ↓ Store(p, 20, mem1)
mem2
```

ИНЖЕНЕРНЫЙ ВЫВОД

Второй Store получает состояние после первого. Порядок записи представлен зависимостью.

Почему нельзя дать обоим Store один `mem0`

ЗАДАЧА

Тогда запись 20 выглядела бы независимой от записи 10.

МЕХАНИЗМ

Но при обращении к одному адресу результат зависит от порядка. `mem1` связывает операции.

ИНЖЕНЕРНЫЙ ВЫВОД

SSA сохраняет необходимые зависимости при оптимизациях.

Load зависит от уже выполненной записи

GO / РАНТАЙМ

```
*p = 10
x := *p
```

ЧТО ПРОИСХОДИТ

В учебной SSA-модели `Store` создает `mem1`; `Load(p, mem1)` должен учитывать именно это состояние.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя прочитать старую память после записи, которую программа должна уже наблюдать.

Вызов функции тоже меняет `Memory`

ЗАДАЧА

Если компилятор не доказал иное, вызванная функция может читать и изменять память.

МЕХАНИЗМ

Вызов обычно становится узлом, который переводит `mem0` в новое состояние `mem1`.

ИНЖЕНЕРНЫЙ ВЫВОД

Непрозрачный вызов ограничивает свободу переставлять операции.

SSA не разрешает любые перестановки

УПРОЩЕНИЕ

Наивно: «Оптимизатор может поменять команды местами ради лучшего конвейера CPU».

ТОЧНАЯ МОДЕЛЬ

Точно: перестановка допустима, только если сохраняется наблюдаемая семантика и зависимости памяти.

ИНЖЕНЕРНЫЙ ВЫВОД

Корректность памяти важнее красоты машинного кода.

Связь с backend под нагрузкой

ЗАДАЧА

Неожиданный вызов или зависимость по памяти может запретить полезное преобразование.

МЕХАНИЗМ

После инлайнинга компилятор видит больше фактов и иногда удаляет этот барьер оптимизации.

ИНЖЕНЕРНЫЙ ВЫВОД

Поэтому цена маленькой функции зависит и от того, виден ли вызывающему ее код.

Семантика сохраняется, проверка исчезает

Проверка допустимого индекса обязательна по смыслу языка, но не обязательно как отдельная инструкция.

```
удаление кода и проверок
prove / BCE
наблюдаем результат в SSA и assembly
```

Преобразований SSA несколько

ЗАДАЧА

Среди важных для курса: удаление недостижимого/мертвого кода, распространение констант, анализ диапазонов, устранение nil-check и проверок границ.

МЕХАНИЗМ

Каждый проход использует уже доказанные факты и может открыть работу следующему.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор — цепочка взаимодействующих доказательств, а не одна «магическая оптимизация».

Удаление мертвого кода

GO / РАНТАЙМ

```
if false {
    expensive()
}
```

ЧТО ПРОИСХОДИТ

Если путь недостижим, ни вызов, ни вычисления внутри него не обязаны попасть в скомпилированную функцию.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор удаляет операции, семантически не влияющие на программу.

Гарантия `panic` и реальная `nil`-проверка

УПРОЩЕНИЕ

Семантика языка требует корректного поведения при ошибочном разыменовании.

ТОЧНАЯ МОДЕЛЬ

Но отдельный `CMP p, 0` может быть лишним, если указатель доказан ненулевым или корректный fault выполняет требуемую проверку.

ИНЖЕНЕРНЫЙ ВЫВОД

Гарантия языка не равна обязательной отдельной инструкции проверки.

Срез индексируется внутри цикла

GO / РАНТАЙМ

```
for i := 0; i < len(s); i++ {
    sum += s[i]
}
```

ЧТО ПРОИСХОДИТ

Проверка `0 <= i < len(s)` для `s[i]` следует из условий самого цикла.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор может удалить повторяющуюся проверку границ.

BCE: путь доказательства

ДОКАЗАТЕЛЬСТВО

```
условие цикла: i < len(s)
↓
допустимый диапазон i
↓
индекс s[i] безопасен
↓
дополнительный check не нужен
```

ИНЖЕНЕРНЫЙ ВЫВОД

Паника при выходе за границы сохраняется: недопустимого пути исполнения здесь нет.

Если доказательство есть — проверка исчезает

УПРОЩЕНИЕ

Упрощение: «`s[i]` каждый раз обязательно исполняет runtime-проверку границ».

ТОЧНАЯ МОДЕЛЬ

Точно: компилятор удаляет check при доказанной безопасности; иначе обязан сохранить корректное поведение.

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость одной операции может измениться без изменения Go-семантики.

Как сломать доказательство BCE

GO / РАНТАЙМ

```
j := transform(i)
sum += s[j]
```

ЧТО ПРОИСХОДИТ

Если `transform` непрозрачна, компилятор может не знать диапазон `j`; тогда проверка остается.

ИНЖЕНЕРНЫЙ ВЫВОД

Доказательство для человека не означает доказательство для текущего анализа компилятора.

Компилятор не читает мысли программиста

ЗАДАЧА

Вам очевидно, что индекс безопасен. Но в графе вычислений недостает установленной связи.

МЕХАНИЗМ

Компилятор сохраняет проверку, потому что не может доказать ее избыточность.

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала найти потерянный факт, потом менять код.

Инструмент для проверок границ

КОМАНДА

```
go build
-gcflags='-d=ssa/check_bce/debug=1
'
.
```

ЧТО ПРОВЕРЯЕМ

Диагностика указывает места с оставшимися `IsInBounds`/родственными проверками.

ИНЖЕНЕРНЫЙ ВЫВОД

ВСЕ проверяется отчетом компилятора, а не догадкой по циклу.

GOSSAFUNC — микроскоп для SSA

GO / РАНТАЙМ

```
GOSSAFUNC=sum go build
```

ЧТО ПРОИСХОДИТ

Появляется `ssa.html`: исходник, начальный SSA, проходы, lowered representation, ассемблер.

ИНЖЕНЕРНЫЙ ВЫВОД

Важнее видеть момент исчезновения проверки, чем только финальный CMP.

Почему `ssa.html` полезнее одного дампа инструкций

АССЕМБЛЕР

Ассемблер отвечает: «что в итоге осталось?».

SSA

SSA по этапам объясняет: «на каком проходе и из-за какого преобразования это изменилось?».

ИНЖЕНЕРНЫЙ ВЫВОД

Исследуем причинность решения компилятора, а не только его результат.

Кто вызывает `growslice`?

В 4.1 мы видели медленный путь. Теперь исследуем, откуда берется проверка емкости.

```
len / cap → условная ветка
быстрая запись или growslice
что видно в профиле CPU
```

Начинаем с одной строки

GO / РАНТАЙМ

```
s = append(s, x)
```

ЧТО ПРОИСХОДИТ

Компилятор должен получить новое `len`, проверить, хватает ли `cap`, и выбрать путь. Вызова `growslice` на каждой операции нет.

ИНЖЕНЕРНЫЙ ВЫВОД

`append` — конструкция языка, а не безусловный вызов функции рантайма.

Принципиальная схема lowering append

ВЕТВЛЕНИЕ

```
oldLen, cap, ptr := s
newLen = oldLen + 1
        ↓
newLen > cap?
├─ нет → записать x
└─ да  → growslice → записать x
        ↓
новый дескриптор среза
```

ИНЖЕНЕРНЫЙ ВЫВОД

Fast path выполняет адресную арифметику и запись. Рост нужен только при недостаточной емкости.

Что происходит, если емкости хватает

ЗАДАЧА

Новый элемент помещается в существующий базовый массив.

МЕХАНИЗМ

Компилятор генерирует адрес ячейки и запись; старая часть массива не копируется и новой аллокации не требуется.

ИНЖЕНЕРНЫЙ ВЫВОД

Большая доля `append` может вообще не заходить в `runtime.growslice`.

Что происходит при исчерпании емкости

УСЛОВИЕ

`newLen > cap`.

ПОСЛЕДСТВИЕ

Вызывается `runtime.growslice`: новая емкость, новый массив, перенос старых данных; после возвращения компилятор записывает добавленный элемент.

ИНЖЕНЕРНЫЙ ВЫВОД

Это тот медленный путь, стоимость которого мы разбирали в 4.1.

Как найти оба пути в машинном коде

АССЕМБЛЕР

```
len + 1
↓ CMP с cap
├─ условный переход на быструю ветку
└─ CALL runtime.growslice
↓ вычисление адреса элемента
↓ STORE x
```

ИНЖЕНЕРНЫЙ ВЫВОД

Конкретные `CMP/Jxx/MOV` зависят от архитектуры; логика выбора остается.

Почему используется беззнаковое сравнение

ЗАДАЧА

Проверка роста должна учитывать не только обычный предел емкости, но и некорректные результаты арифметики при переполнении.

МЕХАНИЗМ

Беззнаковое сравнение позволяет компилятору объединить часть условий.

ИНЖЕНЕРНЫЙ ВЫВОД

Точные SSA-операции и порядок проверок — детали Go 1.27.x.

Два append, но разная подготовка

УПРОЩЕНИЕ

`append(s, e1, e2, e3)` — известное при компиляции число добавляемых элементов.

ТОЧНАЯ МОДЕЛЬ

`append(dst, src...)` может проходить особые преобразования и перенос нескольких элементов.

ИНЖЕНЕРНЫЙ ВЫВОД

Одинаковое имя builtin не гарантирует одинаковый lowering.

Почему профиль `growslice` не равен цене `append`

ЗАДАЧА

В профиле `runtime.growslice` появляются только вызовы медленного пути.

МЕХАНИЗМ

Множество быстрых `append` в этот стек не попадает, хотя в сумме тоже потребляет CPU.

ИНЖЕНЕРНЫЙ ВЫВОД

Интерпретируем профиль через дерево ветвлений, а не по имени API.

Рост среза и хип — тоже не всегда одно

УПРОЩЕНИЕ

Наивно: каждый новый базовый массив `growslice` обязан стать аллокацией в хипе.

ТОЧНАЯ МОДЕЛЬ

В Go 1.27 для некоторых растущих срезов компилятор может оставить промежуточные массивы на стеке и перенести в хип финальное состояние.

ИНЖЕНЕРНЫЙ ВЫВОД

Даже детальная модель рантайма не всегда описывает весь эффект решений компилятора.

Когда компилятор сокращает промежуточные аллокации

ПОЗДНЯЯ ОПТИМИЗАЦИЯ

```
несколько последовательных append
↓ промежуточные базовые массивы
↓ известное время жизни
↓ размещение части данных на стеке
↓ конечный результат при выходе наружу
```

ИНЖЕНЕРНЫЙ ВЫВОД

Конкретная оптимизация зависит от доказуемого времени жизни; это не новая гарантия языка.

Как исследовать append руками

GO / РАНТАЙМ

```
GOSSAFUNC=f go build
go build -gcflags='-S' .
```

ЧТО ПРОИСХОДИТ

В SSA ищем сравнение длины и емкости; в ассемблере — условный переход и при необходимости `CALL runtime.growslice`.

ИНЖЕНЕРНЫЙ ВЫВОД

Цель — понять выбранный путь, а не подсчитать строки исходника.

m[key] до Swiss Table

Структуру таблицы мы уже знаем. Теперь найдем путь от синтаксиса к алгоритму поиска.

```
walk выбирает операцию
тип ключа → специализация
рантайм → хеш / пробирование
```

Чтение из `map`

GO / РАНТАЙМ

```
v := m[key]
v, ok := m[key]
```

ЧТО ПРОИСХОДИТ

Одна форма возвращает значение, другая — значение и признак наличия ключа. Компилятору нужны разные вспомогательные пути.

ИНЖЕНЕРНЫЙ ВЫВОД

Выполняется не «магическая инструкция поиска», а подготовленный вызов рантайма.

От `m[key]` до значения

ПУТЬ ЧТЕНИЯ

```
индексирование m[key]
↓ walk и типизация
↓ mapaccess1* / mapaccess2*
↓ получение слота / результата
↓ загрузка значения
↓ алгоритм Swiss Table
```

ИНЖЕНЕРНЫЙ ВЫВОД

Выбор входа рантайма — задача компилятора; поиск внутри таблицы — задача рантайма.

По типу ключа возможна специализация

ЗАДАЧА

Тип K известен при компиляции.

МЕХАНИЗМ

Для некоторых типов ключа доступны специализированные семейства: целочисленные варианты и строки.

ИНЖЕНЕРНЫЙ ВЫВОД

Рантайму не нужно заново выяснять всю информацию, которую компилятор уже знает.

Ключи `uint64` и `string`

УПРОЩЕНИЕ

`map[uint64]V`: размер и операции над ключом известны; возможен соответствующий быстрый путь.

ТОЧНАЯ МОДЕЛЬ

`map[string]V`: другие представление, хеширование и сравнение; может выбираться отдельная функция.

ИНЖЕНЕРНЫЙ ВЫВОД

Синтаксис поиска один, а стоимость хеша и сравнения зависит от ключа.

Три формы операций `map`

СПЕЦИАЛИЗАЦИЯ

```
чтение        → mapaccess*
присваивание → mapassign*
удаление     → mapdelete*
↓
Swiss Table / группы / пробирование
```

ИНЖЕНЕРНЫЙ ВЫВОД

Операции имеют разную семантику и разные пути рантайма.

Присваивание в `map`

GO / РАНТАЙМ

```
m[key] = value
```

ЧТО ПРОИСХОДИТ

Рантайм находит или создает место значения. Сгенерированный код записывает `value`, учитывая его тип и при необходимости барьер записи GC.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор и рантайм делят работу: найти слот и записать типизированное значение.

Удаление из `map`

GO / РАНТАЙМ

```
delete(m, key)
```

ЧТО ПРОИСХОДИТ

Компилятор выбирает нужный путь удаления. А внутри рантайма работают поиск группы и правила `empty/deleted` из 4.1.

ИНЖЕНЕРНЫЙ ВЫВОД

У одной абстракции чтение, запись и удаление могут иметь разные внутренние маршруты.

Где заканчивается компилятор и начинается Swiss Table

КОМПИЛЯТОР

Компилятор выбирает точку входа по операции и типу.

РАНТАЙМ

Рантайм выбирает таблицу, хеширует ключ, проверяет метаданные групп, выполняет пробирование и сравнение.

ИНЖЕНЕРНЫЙ ВЫВОД

Не смешиваем выбор функции и алгоритм самой хеш-таблицы.

Как увидеть выбор по типу ключа

GO / РАНТАЙМ

```
map[uint64]int
map[string]int
```

ЧТО ПРОИСХОДИТ

Собрать короткие функции поиска и сравнить вызовы/путь ассемблера. Затем проверить предположения о хешировании по 4.1.

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала находим вызванную функцию, потом исследуем ее стоимость.

Кто собирает `Type/ITab + Data`?

После 4.1 мы знаем ABI интерфейса; теперь выясним, какой код строит его два слова.

```
OMAKEFACE / представление значения
аллоцирует ли преобразование?
динамический и прямой вызов
```

Конкретное значение `any`

GO / РАНТАЙМ

```
var x any = v
```

ЧТО ПРОИСХОДИТ

Компилятор должен построить два слова: описание конкретного типа и представление данных. Current `walkConvInterface` формирует `OMAKEFACE`.

ИНЖЕНЕРНЫЙ ВЫВОД

ABI интерфейса появляется как результат преобразования компилятора.

Два слова интерфейса

ПРЕДСТАВЛЕНИЕ

```
конкретный v типа T
↓
слово типа: Type или ITab
слово данных: Data
↓
interface value
```

ИНЖЕНЕРНЫЙ ВЫВОД

Формат из 4.1 обеспечивают совместно компилятор и рантайм.

Откуда берется `Data`

ЗАДАЧА

Некоторые значения могут представляться непосредственно, другим нужна косвенная память.

МЕХАНИЗМ

Текущий путь зависит от конкретного T, времени жизни и выбранной вспомогательной функции преобразования.

ИНЖЕНЕРНЫЙ ВЫВОД

Запись `any(v)` сама по себе не доказывает аллокацию.

Преобразование к интерфейсу и аллокация

УПРОЩЕНИЕ

Плохая модель: любое приведение к `any` обязательно переносит значение в хип.

ТОЧНАЯ МОДЕЛЬ

Точная модель: тип, способ представления, эскейп-анализ и инлайнинг определяют фактическое размещение.

ИНЖЕНЕРНЫЙ ВЫВОД

Измеряем `-m=2` и `allocs/op` для конкретного случая.

Как проверить уход в хип

GO / РАНТАЙМ

```
go build -gcflags='-m=2' .
```

ЧТО ПРОИСХОДИТ

Смотрим, что можно встроить, какое значение выходит за пределы функции и почему.

ИНЖЕНЕРНЫЙ ВЫВОД

Эскейп-диагностика важнее догадки по конструкции `any(v)`.

Косвенный вызов не обязателен

Когда компилятор знает конкретный тип, вызов метода через интерфейс можно упростить.

```
ITab → косвенный вызов
известный тип → прямой вызов
PGO → проверяем горячую ветку
```

Обычный вызов метода через интерфейс

GO / РАНТАЙМ

```
w.Write(buf)
```

ЧТО ПРОИСХОДИТ

Когда динамический тип не известен, используется метод из `ITab`: загрузить адрес из `Fun[]` и сделать косвенный вызов.

ИНЖЕНЕРНЫЙ ВЫВОД

Динамическая диспетчеризация — только один из возможных итогов компиляции.

Тип receiver иногда известен статически

ЗАДАЧА

После анализа программы компилятор доказал, что `w` — конкретный `*bytes.Buffer`.

МЕХАНИЗМ

Тогда вместо `ITab → Fun → косвенный CALL` можно сделать прямой вызов `(*bytes.Buffer).Write`.

ИНЖЕНЕРНЫЙ ВЫВОД

Статическая девиртуализация убирает ненужный уровень динамического выбора.

Косвенный и прямой вызов

УПРОЩЕНИЕ

`w.Write(buf)` при неизвестном T чтение указателя метода из `ITab` косвенный CALL.

ТОЧНАЯ МОДЕЛЬ

Известен T конкретный метод прямой CALL, который иногда можно встроить.

ИНЖЕНЕРНЫЙ ВЫВОД

Экономия часто не в одном переходе, а в оптимизациях после девиртуализации.

Почему девиртуализация открывает инлайнинг

ЦЕПОЧКА ВЫИГРЫША

```
известен конкретный receiver
↓ прямой CALL
↓ можно встроить тело метода
↓ новые константы / диапазоны / время жизни
↓ меньше проверок, веток и аллокаций
```

ИНЖЕНЕРНЫЙ ВЫВОД

Оптимизации усиливают одна другую.

Прямой вызов — еще не инлайнинг

ЗАДАЧА

Компилятор может сделать вызов конкретной функции, но оставить границу функции из-за сложности тела или бюджета инлайнинга.

МЕХАНИЗМ

В таком случае косвенный переход исчез, но тело метода не было встроено.

ИНЖЕНЕРНЫЙ ВЫВОД

Разделяем решения девиртуализации и инлайнинга.

Если тип статически неизвестен, есть PGO

ЗАДАЧА

Профиль выполнения может показать, что 99% вызовов приходятся на один конкретный тип.

МЕХАНИЗМ

Тогда компилятор может добавить проверку этой горячей цели, прямую ветку и запасной путь исходного интерфейсного вызова.

ИНЖЕНЕРНЫЙ ВЫВОД

PGO использует статистику нагрузки, а не доказывает единственный тип.

PGO-девиртуализация: два пути

ПО ПРОФИЛЮ

```
interface call
↓ тип receiver == Foo?
├─ да  → прямой Foo.Method
│         → возможен инлайнинг
└─ нет → исходный косвенный вызов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Специализированная ветка ускоряет частый случай, а общий путь сохраняет семантику.

PGO-оптимизация тоже имеет цену

УПРОЩЕНИЕ

Добавили проверку типа и условный переход перед вызовом.

ТОЧНАЯ МОДЕЛЬ

Если горячий тип действительно доминирует, прямой путь и инлайнинг могут окупить дополнительную проверку.

ИНЖЕНЕРНЫЙ ВЫВОД

Профиль должен соответствовать рабочей нагрузке.

Устаревший PGO-профиль

ЗАДАЧА

Раньше `Foo` был 99% вызовов, теперь — только 20%.

МЕХАНИЗМ

Старая специализированная ветка часто промахивается и переходит к общему косвенному вызову.

ИНЖЕНЕРНЫЙ ВЫВОД

При изменении нагрузки нужно проверять, не перестала ли оптимизация помогать.

Утверждение к конкретному типу

GO / РАНТАЙМ

```
v, ok := x.(User)
```

ЧТО ПРОИСХОДИТ

Частый путь можно свести к проверке динамического указателя типа и выбору ветки; полный поиск `ITab` не всегда нужен.

ИНЖЕНЕРНЫЙ ВЫВОД

Утверждение к конкретному типу отличается от утверждения к интерфейсу.

Утверждение к интерфейсу

GO / РАНТАЙМ

```
w, ok := x.(io.Writer)
```

ЧТО ПРОИСХОДИТ

Здесь требуется проверить реализацию интерфейса; могут использоваться `ITab`, кэш утверждений и вспомогательный путь рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

Одинаковая форма `x.(T)` скрывает разные задачи для concrete и interface `T`.

Два утверждения типа — две разные проверки

УПРОЩЕНИЕ

`x.(User)`: сравнить динамический конкретный тип с `User`.

ТОЧНАЯ МОДЕЛЬ

`x.(io.Writer)`: установить, реализует ли конкретный тип интерфейс и где его методы.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор должен учитывать, является ли целевой `T` интерфейсом.

Как проверить девиртуализацию

GO / РАНТАЙМ

```
go build -gcflags='-m=2' .
GOSSAFUNC=use go build
```

ЧТО ПРОИСХОДИТ

Сравнить статически известный receiver и действительно полиморфный случай: прямой CALL, косвенный CALL, возможный инлайнинг.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя по одному `w.Write` обещать единственный машинный путь.

Кто вызывает `chansend` и `selectgo`?

Теперь соединяем внутреннее устройство каналов из 4.2 с решениями компилятора.

```
отправка / получение → runtime
простые формы select исчезают
общий select → selectgo
```

Отправка в канал

GO / РАНТАЙМ

```
ch <- value
```

ЧТО ПРОИСХОДИТ

Компилятор готовит адрес элемента и данные вызова, затем использует соответствующий путь отправки рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

`chansend` получает `hchan` и адрес, а не исходный синтаксис Go.

Почему общий канал остается механизмом рантайма

ЗАДАЧА

Состояние канала может конкурентно измениться: закрытие, буфер, получатели, ожидания, блокировка.

МЕХАНИЗМ

Компилятор не может статически выбрать прямую передачу или парковку в общем случае.

ИНЖЕНЕРНЫЙ ВЫВОД

Внутри `chansend` остается динамическое дерево решений из 4.2.

Получение из канала

GO / РАНТАЙМ

```
v := <-ch
v, ok := <-ch
```

ЧТО ПРОИСХОДИТ

Компилятор подготавливает место назначения; рантайм определяет, есть ли отправитель, данные в буфере или закрытие.

ИНЖЕНЕРНЫЙ ВЫВОД

Два результата получения и работа `chanrecv` — не отдельная инструкция CPU.

Синтаксис `select` не гарантирует `selectgo`

УПРОЩЕНИЕ

`select` с одним `case` не требует выбора между несколькими операциями.

ТОЧНАЯ МОДЕЛЬ

Общий многовариантный `select` действительно требует массива веток, нескольких локов и алгоритма `selectgo`.

ИНЖЕНЕРНЫЙ ВЫВОД

Классификация формы `select` происходит до входа в рантайм.

`select` без веток

GO / РАНТАЙМ

```
select {}
```

ЧТО ПРОИСХОДИТ

Никакой набор каналов проверять не нужно. Компилятор заменяет форму специализированным путем бесконечного ожидания.

ИНЖЕНЕРНЫЙ ВЫВОД

Не создаем `pollOrder`, `lockOrder` и массив `scase`.

Один `case` — обычная операция

GO / РАНТАЙМ

```
select {
case v := <-ch:
    use(v)
}
```

ЧТО ПРОИСХОДИТ

Здесь нет выбора победителя: можно выполнить простое получение из канала.

ИНЖЕНЕРНЫЙ ВЫВОД

Слово `select` исчезает из общего механизма исполнения.

`case` + `default` — неблокирующий путь

GO / РАНТАЙМ

```
select {
case v := <-ch:
    use(v)
default:
}
```

ЧТО ПРОИСХОДИТ

Компилятор может понизить форму до специализированного неблокирующего получения без общего `selectgo`.

ИНЖЕНЕРНЫЙ ВЫВОД

Количество и типы веток влияют на выбранный путь компиляции.

Когда используется общий `selectgo`

ОБЩИЙ СЛУЧАЙ

```
select с несколькими каналами
↓
компилятор готовит scase[] и порядок
↓
runtime.selectgo
↓
pollOrder / lockOrder / 3 прохода
↓
выбранный case
```

ИНЖЕНЕРНЫЙ ВЫВОД

Механизм 4.2 нужен тогда, когда действительно надо выбирать среди нескольких коммуникаций.

Простой и широкий `select` отличаются по цене

УПРОЩЕНИЕ

Один case: по существу один receive/send.

ТОЧНАЯ МОДЕЛЬ

Много case: подготовка, проверки, порядок локов, регистрация ожиданий и удаление проигравших.

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость нельзя вывести только из слова `select` в исходнике.

Граница функции может исчезнуть

Компилятор получает больше доказуемых фактов, когда видит тело вызванной функции.

```
инлайнинг → константы → BCE / escape
бюджет и размер машинного кода
intrinsics и реальная ISA
```

Главная ценность инлайнинга

ЗАДАЧА

Экономия одного CALL/RET часто незначительна.

МЕХАНИЗМ

После вставки тела вызванной функции компилятор может распространить константы, удалить проверки и точнее определить время жизни объектов.

ИНЖЕНЕРНЫЙ ВЫВОД

Инлайнинг — средство открыть другие оптимизации.

Вспомогательная функция скрывает константу

GO / РАНТАЙМ

```
func limit() int { return 16 }

func f(s []int) int {
if len(s) < limit() { return 0 }
    return s[15]
}
```

ЧТО ПРОИСХОДИТ

После инлайнинга `limit()` превращается в `16`; компилятору проще доказать допустимость `s[15]`.

ИНЖЕНЕРНЫЙ ВЫВОД

Одна граница вызова может скрывать факт, необходимый для ВСЕ.

Инлайнинг влияет на эскейп-анализ

ЗАДАЧА

Если код маленькой функции виден только как непрозрачный вызов, часть сведений о жизни указателей теряется.

МЕХАНИЗМ

После вставки тела компилятор может доказать, что указатель не сохраняется вне нужного кадра.

ИНЖЕНЕРНЫЙ ВЫВОД

Аллокации зависят от места вызова и решений по оптимизации.

Почему компилятор не встраивает все подряд

УПРОЩЕНИЕ

Максимум инлайнинга убирал бы границы вызовов и открывал больше оптимизаций.

ТОЧНАЯ МОДЕЛЬ

Но раздувал бы машинный код, время компиляции и рабочий набор инструкций CPU.

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор использует бюджет; его размер не является гарантией спецификации.

Диагностика инлайнинга

GO / РАНТАЙМ

```
go build -gcflags='-m=2' .
go test -bench=. -benchmem
-gcflags='-l'
```

ЧТО ПРОИСХОДИТ

Первой командой проверяем решение компилятора, второй сравниваем поведение с отключенным инлайнингом.

ИНЖЕНЕРНЫЙ ВЫВОД

Эксперимент отделяет стоимость вызова от вторичных эффектов оптимизаций.

Некоторые вызовы превращаются в intrinsic

ЗАДАЧА

Компилятор знает семантику определенных функций стандартной библиотеки/рантайма.

МЕХАНИЗМ

Вместо обычного CALL он может породить специальную SSA-операцию или машинную последовательность.

ИНЖЕНЕРНЫЙ ВЫВОД

Функция в исходнике не обязательно означает вызов функции в машинном коде.

Intrinsic: где появляется специальная инструкция

ОТ ИСХОДНИКА К CPU

```
исходный вызов известной функции
↓ распознавание компилятором
SSA-операции / переписывания
↓ lower для архитектуры
машинная инструкция / последовательность
```

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор может заменить общий код специализированной инструкцией.

Почему это важно для `matchH2` из 4.1

ЗАДАЧА

Переносимая реализация сравнения контрольных байтов выглядит как арифметика над `uint64`.

МЕХАНИЗМ

Фактический путь для amd64 может использовать более специализированные операции/SIMD — проверяем сгенерированный код.

ИНЖЕНЕРНЫЙ ВЫВОД

Алгоритмическая модель не равна обязательному набору инструкций.

Исходник функции и машинный путь

УПРОЩЕНИЕ

Открыли исходник функции и посчитали выражения, переходы и загрузки.

ТОЧНАЯ МОДЕЛЬ

Но компилятор мог встроить функцию, заменить intrinsic, вычислить константу и удалить часть операций.

ИНЖЕНЕРНЫЙ ВЫВОД

Для производительности нужен SSA/ассемблер конкретной версии Go.

Одна программа — разные архитектуры

Общие доказательства выполняются до того, как операции адаптированы к конкретной ISA.

```
generic SSA → lower
amd64 / arm64
rewrite rules / регистры / стек
```

Общий SSA описывает смысл

ЗАДАЧА

`Add64`, `Load`, `Store`, `Less64` — абстрактные операции.

МЕХАНИЗМ

До архитектурного lowering они не обязаны соответствовать конкретному opcode amd64/arm64.

ИНЖЕНЕРНЫЙ ВЫВОД

Общие оптимизации можно выполнять один раз для разных архитектур.

Развилка при `lower`

АРХИТЕКТУРА

```
общий SSA
       ↓
       lower
     /       \n  amd64    arm64
     ↓         ↓
свои операции / свои инструкции
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одинаковый Go-код проходит разные физические пути на разных ISA.

amd64 и arm64 по-разному обращаются к памяти

УПРОЩЕНИЕ

amd64 допускает многие операции непосредственно с операндом из памяти.

ТОЧНАЯ МОДЕЛЬ

arm64 чаще использует отдельную загрузку, арифметическую операцию и запись; у ISA свои режимы адресации.

ИНЖЕНЕРНЫЙ ВЫВОД

Сравнивать только число инструкций разных ISA нельзя.

Зачем отдельный нижний слой

ЗАДАЧА

Факт `x + 0 = x` верен на всех CPU; его полезно доказать до выбора архитектуры.

МЕХАНИЗМ

А объединение загрузки с операцией зависит от наборов инструкций amd64/arm64.

ИНЖЕНЕРНЫЙ ВЫВОД

Отделяем математику оптимизации от возможностей конкретного процессора.

Правила переписывания SSA

ЗАДАЧА

Множество локальных преобразований можно задать как образец и замену, например `x + 0 → x`.

МЕХАНИЗМ

Это удобнее, чем огромная серия специальных `if` для каждого сочетания операций.

ИНЖЕНЕРНЫЙ ВЫВОД

Простые правила дополняют, но не заменяют сложный анализ диапазонов.

Несколько переписываний подряд

ПРАВИЛА

```
(x + 0) * 1
↓ убрать + 0
x * 1
↓ убрать * 1
x
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одно преобразование создает возможность для следующего; passes применяются до нужного устойчивого результата.

`prove` и rewrite rules решают разные задачи

УПРОЩЕНИЕ

Rewrite: локально распознать выражение вроде `x & 0`.

ТОЧНАЯ МОДЕЛЬ

`prove`: понять диапазоны значения через ветвления и циклы, чтобы удалить проверки.

ИНЖЕНЕРНЫЙ ВЫВОД

Не все полезные факты помещаются в одну локальную подстановку.

SSA-значений намного больше, чем регистров

ЗАДАЧА

У функции могут быть десятки и сотни одновременно живых SSA-значений.

МЕХАНИЗМ

CPU предоставляет гораздо меньше физических регистров. Требуется решение о размещении.

ИНЖЕНЕРНЫЙ ВЫВОД

После выбора ISA приходит распределение регистров.

Время жизни значения влияет на давление на регистры

ЗАДАЧА

Значение определяется в одном месте, а последний раз используется намного позже.

МЕХАНИЗМ

Чем больше таких интервалов пересекаются, тем выше давление на доступные регистры.

ИНЖЕНЕРНЫЙ ВЫВОД

Стоимость сложного кода иногда скрыта в вынужденных обращениях к стеку.

Spill: когда регистры закончились

РАЗМЕЩЕНИЕ

```
значение вычислено
↓ регистров не хватает
STORE в стековый слот
↓ ... другие операции ...
LOAD обратно в регистр
↓ использовать значение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Spill добавляет память и зависимости, которых не было видно в исходном Go.

Переменная Go — не один регистр

УПРОЩЕНИЕ

`x := 42` выглядит как локальная переменная с конкретным именем.

ТОЧНАЯ МОДЕЛЬ

После компиляции `x` может стать константой, исчезнуть, жить в разных регистрах или иметь стековый слот.

ИНЖЕНЕРНЫЙ ВЫВОД

Имена переменных не определяют физическое размещение данных.

Стековый кадр формируется после оптимизаций

ЗАДАЧА

Компилятор знает локальные слоты, spill-слоты, служебные данные и правила ABI.

МЕХАНИЗМ

Собирается конкретная раскладка памяти для вызова функции.

ИНЖЕНЕРНЫЙ ВЫВОД

Стековый кадр — физическая структура, а не список локальных переменных исходника.

GC требует информацию от компилятора

ПРОБЛЕМА

Машинное слово само не говорит, содержит ли оно живой Go-указатель.

РЕШЕНИЕ

Компилятор рассчитывает живость указателей в безопасных точках GC и сохраняет метаданные для сканирования стека.

ИНЖЕНЕРНЫЙ ВЫВОД

Точный GC Go невозможен без участия компилятора.

От типов до точного сканирования стека

МОСТ К GC

```
Go-типы
↓ анализ компилятора
стековый кадр + живость указателей
↓ GC safe point
рантайм сканирует только нужные корни
```

ИНЖЕНЕРНЫЙ ВЫВОД

Точная живость не дает GC удерживать давно мертвые объекты только из-за старых битов в слоте.

`obj.Prog` и ассемблер

ЗАДАЧА

После архитектурного SSA компилятор создает внутренний поток машинно-зависимых инструкций `obj.Prog`.

МЕХАНИЗМ

Ассемблер формирует объектный машинный код.

ИНЖЕНЕРНЫЙ ВЫВОД

Это граница от модели значений/блоков к конкретным инструкциям CPU.

Что нужно уметь читать в ассемблере

УПРОЩЕНИЕ

Не требуется знать все инструкции ISA или писать backend на Go assembly.

ТОЧНАЯ МОДЕЛЬ

Достаточно находить CALL/RET, сравнение и переходы, загрузку/запись, расчет адреса и имена функций рантайма.

ИНЖЕНЕРНЫЙ ВЫВОД

Смотрим не «все инструкции», а проверяем конкретную гипотезу.

От профиля к SSA и ассемблеру

Каждая команда отвечает на конкретный вопрос. Читаем машинный код после измерения.

```
-m=2 / GOSSAFUNC
-S / objdump / BCE
профиль → гипотеза → проверка
```

`-m=2`: инлайнинг и эскейп-анализ

GO / РАНТАЙМ

```
go build -gcflags='-m=2' .
```

ЧТО ПРОИСХОДИТ

В выводе ищем, какие функции встраиваются, какие значения уходят в хип и почему.

ИНЖЕНЕРНЫЙ ВЫВОД

Диагностика компилятора отвечает на вопрос о решении, а не о фактической CPU-нагрузке.

GOSSAFUNC: увидеть путь преобразований

GO / РАНТАЙМ

```
GOSSAFUNC=foo go build
```

ЧТО ПРОИСХОДИТ

В файле `ssa.html` можно сравнить значения и блоки до и после проходов SSA, а затем увидеть итоговые инструкции.

ИНЖЕНЕРНЫЙ ВЫВОД

Это лучший инструмент для вопроса «почему эта проверка исчезла?».

-S: посмотреть сгенерированный код

GO / РАНТАЙМ

```
go build -gcflags='-S' .
```

ЧТО ПРОИСХОДИТ

В небольшом примере можно найти `CALL runtime.growslice`, косвенный вызов, `CMP`, переходы и записи.

ИНЖЕНЕРНЫЙ ВЫВОД

Не начинаем с огромного дампа всего сервиса.

`go tool compile -S` не равен `go build`

GO / РАНТАЙМ

```
go tool compile -S file.go
```

ЧТО ПРОИСХОДИТ

Прямой компилятор удобен для автономной функции. Обычный `go build` дополнительно учитывает пакеты, импорт, кэш и сборочный контекст.

ИНЖЕНЕРНЫЙ ВЫВОД

Для сравнения производительности нужно фиксировать способ сборки.

`objdump`: уже готовый бинарник

GO / РАНТАЙМ

```
go tool objdump -s 'main.foo'
./app
```

ЧТО ПРОИСХОДИТ

Проверяем инструкции конкретного символа уже внутри готового исполняемого файла.

ИНЖЕНЕРНЫЙ ВЫВОД

Когда важен фактический артефакт, смотрим объектный код, а не только исходник.

Отчет об оставшихся проверках

GO / РАНТАЙМ

```
go build
-gcflags='-d=ssa/check_bce/debug=1
'
.
```

ЧТО ПРОИСХОДИТ

Компилятор показывает места, где проверка допустимости индекса не была удалена.

ИНЖЕНЕРНЫЙ ВЫВОД

Ищем потерянное доказательство, а не «оптимизируем» цикл вслепую.

Диагностика конкретного SSA-прохода

GO / РАНТАЙМ

```
go tool compile -d ssa/help
```

ЧТО ПРОИСХОДИТ

Список отладочных возможностей компилятора. Используем, если `GOSSAFUNC` недостаточно.

ИНЖЕНЕРНЫЙ ВЫВОД

Не превращаем специальные debug-флаги в обычные параметры production-сборки.

Порядок реального расследования

ДИАГНОСТИКА

```
симптом под нагрузкой
↓ CPU / alloc profile
горячая функция / место в исходнике
↓ небольшой воспроизводимый пример
-m=2 → SSA → ассемблер
↓ изменить код → измерить снова
```

ИНЖЕНЕРНЫЙ ВЫВОД

Начинаем с ограниченного ресурса и измерения, а не с поиска «страшного MOV».

Go компилятор рантайм CPU

Собираем цепочки уже изученных конструкций без повторения самих алгоритмов рантайма.

```
append / map / interface
chansend / selectgo
машинный код в конце
```

Цепочка append целиком

APPEND

```
s = append(s, x)
↓ проверка нового len и cap
├─ место есть → запись в массив
└─ места нет → runtime.growslice
↓ вычислить адрес / сохранить срез
↓ lower / машинный код
```

ИНЖЕНЕРНЫЙ ВЫВОД

Теперь понятно, кто решил вызвать `growslice`, прежде чем начался сам рост массива.

Цепочка `map` целиком

MAP

```
v := m[key]
↓ тип key + операция чтения
↓ mapaccess*
↓ хеш ключа
↓ группы Swiss Table / probing
↓ сравнение ключа → значение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор выбирает помощник рантайма, рантайм выполняет алгоритм таблицы.

Цепочка вызова интерфейса

INTERFACE

```
w.Write(buf)
↓ известен конкретный receiver?
├─ да → прямой вызов → инлайнинг?
└─ нет → ITab → косвенный вызов
↓ SSA → lower → машинный код
```

ИНЖЕНЕРНЫЙ ВЫВОД

Одна и та же запись может породить совершенно разные вызовы.

Цепочка отправки в канал

CHANNEL

```
ch <- value
↓ преобразование компилятора
↓ runtime.chansend
↓ получатель? буфер? ожидание?
↓ scheduler / копирование / GC
```

ИНЖЕНЕРНЫЙ ВЫВОД

Компилятор готовит вызов, а текущее состояние канала определяет его реальную стоимость.

Цепочка `select` целиком

SELECT

```
select в исходнике
↓ компилятор классифицирует форму
0 case → ожидание
1 case → обычный send/receive
case + default → неблокирующий путь
несколько case → runtime.selectgo
```

ИНЖЕНЕРНЫЙ ВЫВОД

Механизм `pollOrder/lockOrder` нужен только после общего lowering.

Четыре одинаково короткие строки — разная цена

УПРОЩЕНИЕ

`append(s, x)` — сравнение и запись; иногда рост. `m[key]` — хеш и поиск.

ТОЧНАЯ МОДЕЛЬ

`w.Write` — прямой/косвенный вызов. `ch <- v` — конкурентный протокол канала.

ИНЖЕНЕРНЫЙ ВЫВОД

Объем исходного текста почти ничего не говорит о стоимости исполнения.

Версия Go — часть профиля производительности

ЗАДАЧА

В исходниках ничего не меняли, но обновили toolchain.

МЕХАНИЗМ

Могут измениться решения об инлайнинге, escape, BCE, intrinsics, вспомогательные функции и размещение значений.

ИНЖЕНЕРНЫЙ ВЫВОД

После обновления измеряем приложение и сравниваем решение компилятора.

Запустить исследовать измерить

Небольшие воспроизводимые эксперименты проверяют гипотезы из лекции на конкретном Go.

```
append / ВСЕ / map / интерфейс
select / архитектуры / intrinsics
сравнение сборок и профилей
```

Эксперимент 1: append с емкостью и без

GO / РАНТАЙМ

```
s := make([]int, 0, 1024)
s = append(s, x)
```

ЧТО ПРОИСХОДИТ

Сравнить с ростом из нулевой емкости. В SSA/ассемблере найти проверку и медленный путь. Измерить аллокации.

ИНЖЕНЕРНЫЙ ВЫВОД

Доказать, что `append` не обязан каждый раз вызывать `growslice`.

Эксперимент 2: убрать проверку границ

GO / РАНТАЙМ

```
for i := 0; i < len(s); i++ {
    total += s[i]
}
```

ЧТО ПРОИСХОДИТ

Сравнить с индексом из непрозрачной функции; использовать `ssa/check_bce`, `GOSSAFUNC` и сборку `-S`.

ИНЖЕНЕРНЫЙ ВЫВОД

Увидеть, как доказательство диапазона удаляет инструкцию проверки.

Эксперимент 3: выбрать `map`-путь

ЗАДАЧА

Сделать одинаковые операции на `map[uint64]int` и `map[string]int`.

МЕХАНИЗМ

Проверить вызываемые вспомогательные функции, а затем связать результат со стоимостью хеша и сравнения.

ИНЖЕНЕРНЫЙ ВЫВОД

Тип ключа известен компилятору и влияет на внутренний путь.

Эксперимент 4: девиртуализировать интерфейс

ЗАДАЧА

Один случай — конкретный receiver известен, другой допускает два разных типа.

МЕХАНИЗМ

Сравнить `-m=2`, SSA и прямые/косвенные вызовы; проверить, появился ли инлайнинг.

ИНЖЕНЕРНЫЙ ВЫВОД

Доказать, что `w.Write` не означает неизбежный вызов через `ITab`.

Эксперимент 5: отключить инлайнинг

GO / РАНТАЙМ

```
go test -bench=. -benchmem
go test -bench=. -benchmem
-gcflags='-l'
```

ЧТО ПРОИСХОДИТ

Сравнить число аллокаций и задержки; подтвердить изменение по диагностике компилятора.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена границы функции может проявляться через другие оптимизации.

Эксперимент 6: формы `select`

ЗАДАЧА

Сравнить `select {}`, один `case`, `case + default` и два канала.

МЕХАНИЗМ

Найти, какие формы идут напрямую к send/receive, а какая использует `selectgo`.

ИНЖЕНЕРНЫЙ ВЫВОД

Синтаксис `select` еще не означает полный алгоритм выбора.

Эксперимент 7: amd64 и arm64

ЗАДАЧА

Собрать одну функцию с арифметикой по срезу под две архитектуры.

МЕХАНИЗМ

Сравнить lowered SSA и ассемблер; не делать вывод о скорости только из числа инструкций.

ИНЖЕНЕРНЫЙ ВЫВОД

Generic SSA сохраняет смысл, а `lower` учитывает ISA.

Эксперименты 8–9: intrinsic и Swiss Table

ЗАДАЧА

Найти в текущем Go intrinsic-путь; отдельно сравнить исходный `matchH2` с конечными инструкциями поиска в `map`.

МЕХАНИЗМ

Зафиксировать версию и не обещать одинаковые инструкции на всех CPU.

ИНЖЕНЕРНЫЙ ВЫВОД

Исходник вспомогательной функции и реальный машинный путь могут различаться.

Эксперимент 10: PGO-девиртуализация

ЗАДАЧА

Создать профиль с доминирующим concrete receiver и сборки с PGO и без него.

МЕХАНИЗМ

Сравнить условную прямую ветку, общий косвенный путь и benchmark.

ИНЖЕНЕРНЫЙ ВЫВОД

PGO выгоден только если профиль представляет реальную нагрузку.

Эксперимент 11: сравнить версии Go

ЗАДАЧА

Одна и та же функция под двумя версиями toolchain.

МЕХАНИЗМ

Сравнить `-m=2`, SSA, ассемблер и измеренную стоимость. Найти конкретное изменившееся решение.

ИНЖЕНЕРНЫЙ ВЫВОД

Фраза «новая Go быстрее» не заменяет механистического объяснения.

Контракт языка и решения компилятора

После каждой оптимизации спрашиваем: что гарантировано, что является реализацией и где цена?

```
язык → семантика
compiler → возможный машинный путь
профиль → фактическая стоимость
```

Гарантии языка и детали компилятора

КОНТРАКТ

Язык гарантирует семантику `append`, `map`, interfaces, каналов, `select` и panic при выходе за границы.

РЕАЛИЗАЦИЯ

Не гарантирует конкретный IR, SSA-проход, инлайнинг, helper, register allocation или число машинных инструкций.

ИНЖЕНЕРНЫЙ ВЫВОД

Для корректности опираемся на язык и API, для диагностики — на конкретную версию реализации.

Симптом: неожиданная аллокация

ЗАДАЧА

В профиле аллокаций появилась горячая точка.

МЕХАНИЗМ

Проверяем `-m=2`, место выхода указателя, инлайнинг и преобразование к интерфейсу; после правки повторяем `benchmem`.

ИНЖЕНЕРНЫЙ ВЫВОД

Механизм аллокации доказывают отчет компилятора и измерение.

Симптом: маленькая функция потребляет CPU

ЗАДАЧА

По профилю горячая функция выглядит короткой.

МЕХАНИЗМ

Проверяем вызов рантайма, косвенный метод, копирование большого значения, оставшиеся проверки и инлайнинг.

ИНЖЕНЕРНЫЙ ВЫВОД

Маленькая функция исходника может порождать заметную работу.

Симптом: изменилась скорость после обновления Go

ЗАДАЧА

Бизнес-код прежний, а p99 или `ns/op` изменились.

МЕХАНИЗМ

Сравниваем профили и отчеты компилятора, затем SSA и ассемблер на горячей функции.

ИНЖЕНЕРНЫЙ ВЫВОД

Версия toolchain — реальная составляющая стоимости backend-кода.

Когда в SSA не стоит углубляться

УПРОЩЕНИЕ

Если 90% времени запрос ожидает PostgreSQL, машинный код обработчика не главный ограничитель.

ТОЧНАЯ МОДЕЛЬ

Если профиль показывает горячую вычислительную функцию и неожиданную работу компилятора — SSA уже полезен.

ИНЖЕНЕРНЫЙ ВЫВОД

Начинаем с ограниченного ресурса и измерения, затем открываем нужный технический слой.

Финальная модель 4.3

ИТОГОВАЯ ЦЕПОЧКА

```
Go-код → типы / IR / middle end
↓ walk → SSA / prove / BCE
↓ lower: amd64 / arm64
↓ регистры / стек / живость указателей
↓ obj.Prog / ассемблер / машинный код
↓ CPU / рантайм → измерение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Теперь весь путь от конструкции Go до физической работы можно расследовать последовательно.

Главный инженерный вывод

ИСХОДНИК

Снаружи: `append`, `m[key]`, `w.Write`, `select` — короткие конструкции.

ИСПОЛНЕНИЕ

Внутри: компилятор доказывает свойства, удаляет проверки, выбирает вспомогательные пути и опускает код до ISA.

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала измеряем горячий путь, затем изучаем решение компилятора и снова измеряем.

Следующий слой уже за пределами 4.3

ГРАНИЦА

Мы получили машинный код отдельной функции.

ПРОДОЛЖЕНИЕ

Дальше возникают ABI вызовов, generics codegen и linker: как функции вызывают друг друга и собираются в исполняемый файл.

ИНЖЕНЕРНЫЙ ВЫВОД

Это отдельная 4.4, а не продолжение анализа SSA внутри этой лекции.